This is where a software called 'scheduler' comes into the picture—to schedule running processes to CPU(s) time. Thus, it is the scheduler that makes it possible for programs to run concurrently in an operating system.

There are several terms associated with a process scheduler—some of them, to quote Wikipedia, are:

- CPU utilisation: To keep the CPU as busy as possible.
- Throughput: The number of processes that complete their execution per time unit.
- Turnaround: The total time between submission of a process and its completion.
- Waiting time: The amount of time a process has been waiting in the ready queue.
- Response time: The amount of time it takes from when a request was submitted until the first response is produced.
- Fairness: Equal CPU time to each thread.

Jargon apart, the purpose of a scheduler is to always provide a responsive user experience by allocating tasks to the CPU efficiently.

Linux schedulers

Linux has had a multi-level feedback queue with priority levels ranging from 0-140 since version 2.5. Priority levels in the range 0-99 are reserved for real-time tasks and 100-140 are considered nice task levels. From versions 2.6.0 to 2.6.23, Linux used an O(1) scheduler that could schedule processes within a constant amount of time, regardless of how many processes are running on the operating system. When kernel version 2.6.23 was released, developers replaced this scheduler with the Completely Fair Scheduler that uses Red Black trees instead of queues.

Completely Fair Scheduler

Ingo Molnar developed the Completely Fair Scheduler, inspired by Kolivas' implementation of fair scheduling called Rotating Staircase Deadline. Quoting Wikipedia, "CFS is an implementation of a well studied, classic algorithm called fair queuing. CFS does not consider a task to be a sleeper if it sleeps for very short time—a short sleeper might be entitled to some bonus time, but never more than it would have had, had it not slept. CFS has a scheduling complexity of O(log N) where N is the number of tasks in the run queue."

Run queues in CFS are implemented as Red Black Trees [*en.wikipedia.org/wiki/Red-black_tree*]. Because of this implementation, the selection of a process for CPU allocation can be done in constant time, but reinserting the job after it has run needs O(log N) operations.

The name 'Completely Fair Scheduler' is technically incorrect as the algorithm only guarantees the 'unfair' level to be less than O(n) where n is the number of processes. There are more complicated algorithms that can perform well at 'unfair' levels—for example, O(log n). CFS helps assure every process to get its fair share of CPU time.

Levels of scheduling

In multi-tasking systems, schedulers operate at three levels based on the relative frequency with which they perform their functions. The different levels of scheduling are:

- **Long-term scheduling:** The long-term, or admission, scheduler decides which programs are admitted to the system for execution and when, and which ones should be exited. In modern operating systems, this is used to make sure that real-time processes get enough CPU time to finish their tasks. Long-term scheduling is also important in large-scale systems such as batch processing systems, computer clusters, super computers and render farms.
- **Mid-term scheduling:** The mid-term scheduler makes use of virtual memory, by temporarily removing processes from main memory and places them on secondary memory (such as a disk drive) or vice versa. This technique is known as "swapping out" or "swapping in". The mid-term scheduler decides which processes are needed to be swapped by finding out the processes which have not been active for some time, or a process which has a low priority, or a process which is page faulting frequently, or a process which is taking up a large amount of memory in order to free up main memory for other processes, swapping the process back in later when more memory is available, or when the process has been unblocked and is no longer waiting for a resource.
- **Short-term scheduling:** The short-term scheduler or dispatcher decides which of the ready, in-memory processes are to be executed next and for how long. In this way, the short-term scheduler makes scheduling decisions much more frequently than the long-term or mid-term schedulers. This scheduler can be preemptive, which means that it can forcibly remove processes from a CPU when it wants to allocate that CPU to another process, or non-preemptive—in which case the scheduler is unable to forcibly deallocate the processes off the CPU.

–Courtesy: Wikipedia [*en.wikipedia.org/wiki/Scheduling_(computing)*]

What's with this BFS?

The xkcd.com comic 'Supported_Features' [Figure 1; *http://xkcd.com/619/*], which picked on Linux for having support for 4,096 CPUs while not supporting essential desktop computing features like smooth flicker-free full screen Flash videos, inspired Kolivas to start working on a new scheduler to address the requirements of desktop users.

Kolivas wrote BFS to improve responsiveness on non-uniform memory access (NUMA) Linux desktop computers